STAT 412/612 Week 11: Midterm 2 (40 pts)

Solutions

Your Name

Instructions

- Please place your answers in an R Markdown document.
- Submit this R Markdown document, and the resulting knitted PDF, to Blackboard before the exam time has expired.
- Make sure all code is visible in the PDF by making sure `knitr::opts_chunk$set(echo = TRUE)` is in your first code chunk.
- You have two-hours and thirty minutes.
- You may use your notes and the course website. No Google, other websites, or LLMs.
- You may not discuss this exam with anyone (including over instant message or text).
- Use the tidyverse, not base R, where possible. Too much base R will result in points lost.

Question 1: NBA (8 pts)

The following are small data frames of statistics from the 2019 NBA season. The variables in `teams` are:

- `Team`: The name of the team.
- `W`: The total number of wins.
- `L`: The total number of losses.
- `PSG`: The average number of points per game.
- `PAG`: The average number of points allowed per game.

The variables in `players` are:

- `Name`: The name of the player.
- `teamID`: The team of the player.
- `Age`: Age of the player.
- `G`: Number of games played.
- `GS`: Number of games started.
- `MP`: Minutes played per game.
- `FG`: Average number of field goals per game.
- `FGA`: Average number of field goal attempts per game.

The variables in `games` are:

- `date_time`: The date and start-time of the game.
- `location`: The location of the game.
- `bulls`: The Bulls' points.
- `cavs`: The Cavaliers' points.

```
library(tidyverse)
```

```
teams <- tribble(~Team,       ~W, ~L, ~PSG, ~PAG,
                 ##----------/---/---/-----/-----
                 "Cavaliers", 17, 53, 104,  113,
                 "Bulls",     19, 52, 105,  113)
```

```
players <- tribble(~Name,             ~teamID,     ~Age, ~G, ~GS, ~MP,  ~FG, ~FGA,
                   ##---------------/------------/-----/---/----/-----/----/-----
                   "Zach LaVine",     "Bulls",     23,   62, 61,  34.5, 8.5, 18.1,
                   "Lauri Markkanen", "Bulls",     21,   48, 47,  32.7, 6.6, 15.2,
                   "Otto Porter",     "Bulls",     25,   15, 15,  32.8, 6.5, 13.4,
                   "Kevin Love",      "Cavaliers", 30,   16, 16,  27.4, 5.0, 13.0,
                   "Jordan Clarkson", "Cavaliers", 26,   69, 0,   27.1, 6.5, 14.7,
                   "Collin Sexton",   "Cavaliers", 20,   70, 60,  31.0, 6.0, 14.3)
```

```
games <- tribble(~date_time,             ~location,   ~bulls, ~cavs,
                 ##--------------------/------------/-------/------
                 "Nov 10, 2018  8:00pm", "Bulls",     99,     98,
                 "Dec 23, 2018  6:00pm", "Cavaliers", 112,    92,
                 "Jan 21, 2019  1:00pm", "Cavaliers", 104,    88,
                 "Jan 27, 2019  3:30pm", "Bulls",     101,    104)
```

1. (4 pts) Use the `parse_datetime()` function from `readr` to parse `date_time` in `games` to be a date-time variable and assign it back to `games`.

```
games %>%
  mutate(date_time = parse_datetime(date_time, "%b %d, %Y  %H:%M%p")) ->
  games

select(games, date_time)
```

```
## # A tibble: 4 x 1
##   date_time
##   <dttm>
## 1 2018-11-10 20:00:00
## 2 2018-12-23 18:00:00
## 3 2019-01-21 13:00:00
## 4 2019-01-27 15:30:00
```

2. (4 pts) Add the data from the `teams` data frame to the `players` data frame.

```
players %>%
  left_join(teams, by = c("teamID" = "Team")) ->
  players
head(players,10)
```

```
## # A tibble: 6 x 12
##   Name   teamID   Age     G    GS    MP    FG   FGA     W     L   PSG   PAG
##   <chr>  <chr>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Zach ~ Bulls     23    62    61  34.5   8.5  18.1    19    52   105   113
```

```
## 2 Lauri~ Bulls     21    48    47  32.7   6.6  15.2    19    52   105   113
## 3 Otto ~ Bulls     25    15    15  32.8   6.5  13.4    19    52   105   113
## 4 Kevin~ Caval~    30    16    16  27.4   5    13      17    53   104   113
## 5 Jorda~ Caval~    26    69     0  27.1   6.5  14.7    17    53   104   113
## 6 Colli~ Caval~    20    70    60  31     6    14.3    17    53   104   113
```

Question 2: Strings (10 pts)

For this question, **use only functions from the tidyverse**.

Consider the following data frame:

```
dfdat <- tribble(~frac,  ~words,
                 ##-----/---------
                 "4/5",  "Not",
                 "7/8",  "all",
                 "2/3",  "those",
                 "4/7",  "who",
                 "2/11", "wander",
                 "8/13", "are",
                 "4/6",  "lost.")
```

Your solutions should be general and not specific to these strings.

1. (4 pts) Convert the variable `frac` from a character vector of fractions to a numeric vector of decimal equivalences. For example, `"4/5"` should become `0.8`. (Hint: No stringr functions or regular expressions are needed here).

```
dfdat %>%
  separate(col = frac, into = c("num", "den"), sep = "/") %>%
  mutate(num = parse_number(num),
         den = parse_number(den),
         dec = num / den) ->
  dfdat

select(dfdat, dec)
```

```
## # A tibble: 7 x 1
##     dec
##   <dbl>
## 1 0.8
## 2 0.875
## 3 0.667
## 4 0.571
## 5 0.182
## 6 0.615
## 7 0.667
```

2. (4 pts) Use a **single regular expression** to find all words in `words` that begin with "wh" or "wa" and replace those patterns with "te". So "who" should go to "teo" and "wander" should go to "tender".

```
dfdat %>%
  mutate(newwords = str_replace(words, "^w(h|a)", "te")) %>%
  select(newwords)
```

```
## # A tibble: 7 x 1
##   newwords
##   <chr>
## 1 Not
## 2 all
## 3 those
## 4 teo
## 5 tender
## 6 are
## 7 lost.
```

3. (2 pts) **Challenge** (do this one last). Pig Latin is a word modification where the first consonant cluster (groupings of consonants and no vowels) is moved to the end of the word before appending "ay" to the end of the word. For example, the pig Latin version of "hello" is "ellohay", the pig Latin version of "study" is "udystay", the pig Latin version of "aloe" is "aloeay". Create the pig Latin versions of the strings in the `words` variable in the `dfdat` data frame.

   For half a point extra credit: Make sure capitalized words remain capitalized. For example, the pig latin version of "John" should be "Ohnjay", not "ohnJay". Also, respect puncutation so that "goodbye." is "oodbyegay.", not "oodbye.gay".

```
## Full solution (with extra credit)
dfdat %>%
  mutate(iscap = str_detect(words, "^[A-Z]"),
         isperiod = str_detect(words, "\\.$")) %>%
  mutate(pig = str_replace(words, "(^[^aeiou]*)(.*)", "\\2\\1")) %>%
  mutate(pig = str_c(pig, "ay")) %>%
  mutate(pig = str_replace(pig, "\\.", ""),
         pig = ifelse(isperiod, str_c(pig, "."), pig)) %>%
  mutate(pig = str_to_lower(pig),
         pig = ifelse(iscap, str_replace(pig, "^.",
                                         str_to_upper(str_extract(pig, "^."))),
                      pig)) ->
  dfdat
select(dfdat, pig)
```

```
## # A tibble: 7 x 1
##   pig
##   <chr>
## 1 Otnay
## 2 allay
## 3 osethay
## 4 owhay
## 5 anderway
## 6 areay
## 7 ostlay.
```

```
## Simpler solution (without extra credit)
dfdat %>%
  mutate(pig = str_replace(words, "(^[^aeiou]*)(.*)", "\\2\\1")) %>%
  mutate(pig = str_c(pig, "ay")) %>%
  select(pig)
```

```
## # A tibble: 7 x 1
##   pig
##   <chr>
## 1 otNay
## 2 allay
## 3 osethay
## 4 owhay
## 5 anderway
## 6 areay
## 7 ost.lay
```

Question 3: Tidying Simulation Output (22 pts)

The file "mse_mat.csv" contains a subset of real output from a simulation study. The first six variables are the conditions being changed in the simulation study. Each row is one iteration of the simulation study. The condition variables are `current_seed`, `nullpi`, `Nsamp`, `ncontrols`, `cor`, and `poisthin`. It's not important to know what those conditions are except for:

- `cor`: A string, containing either a single value in parentheses or two values in parentheses separated by a comma and a space. For example `"(0.5)"` and `"(0.25, 0.25")` are two possible values.

The remainder of the variables represent a numerical evaluation (called the "mean squared error" or "mse") of some combination of "mean method" and "variance method". There are nine mean methods: `ols`, `ruv2`, `ruv3`, `ruv4`, `cate`, `ruvbn`, `ruvbnl`, `ruvbnn`, and `ruvb`.

There are six variance methods: `o`, `m`, `c`, `l`, `lm`, and `lc`.

It's not important to know what those methods actually do. Not all mean methods have a separate variance method.

The mean method and variance method are separated by an underscore `"_"`. For example, the `ols_m` column contains the mse calculated using the "ols" mean method and the "m" variance method, while the column `ruvb_` contains the mse calculated using the "ruvb" mean method with no separate variance method.

These data are messy. Let's tidy them.

For each part, print out the first six rows of your result using `head()`.

For this question, the *only* non tidyverse functions you may use are are `head()` and `c()`.

1. (2 pts) Use readr to load these data into R and view the first six rows.

```
msemat <- read_csv("./mse_mat.csv")
```

```
## Parsed with column specification:
## cols(
##   .default = col_double(),
##   cor = col_character(),
##   poisthin = col_logical()
## )
```

```
## See spec(...) for full column specifications.
```

```
head(msemat)
```

```
## # A tibble: 6 x 52
##   current_seed nullpi Nsamp ncontrols cor   poisthin ols_o ols_m ols_c
##          <dbl>  <dbl> <dbl>     <dbl> <chr> <lgl>    <dbl> <dbl> <dbl>
## 1         2223    0.9    20       100 (0.2~ TRUE     0.197 0.197 0.197
## 2         2223    0.9     6       100 (0.5) TRUE     0.460 0.460 0.460
## 3         2223    0.9     6       100 (0.5~ TRUE     0.664 0.664 0.664
## 4         2224    0.9     6       100 (0.2~ TRUE     0.433 0.433 0.433
## 5         2224    0.9    20       100 (0.5~ TRUE     0.182 0.182 0.182
## 6         2224    0.9    10       100 (0.5~ TRUE     0.449 0.449 0.449
## # ... with 43 more variables: ols_l <dbl>, ols_lm <dbl>, ols_lc <dbl>,
## #   ruv2_o <dbl>, ruv2_m <dbl>, ruv2_c <dbl>, ruv2_l <dbl>, ruv2_lm <dbl>,
## #   ruv2_lc <dbl>, ruv3_o <dbl>, ruv3_m <dbl>, ruv3_c <dbl>,
## #   ruv3_lb <dbl>, ruv3_lbm <dbl>, ruv3_lbc <dbl>, ruv3_la <dbl>,
## #   ruv3_lam <dbl>, ruv3_lac <dbl>, ruv4_o <dbl>, ruv4_m <dbl>,
## #   ruv4_c <dbl>, ruv4_l <dbl>, ruv4_lm <dbl>, ruv4_lc <dbl>,
## #   cate_o <dbl>, cate_m <dbl>, cate_c <dbl>, cate_lb <dbl>,
## #   cate_lbm <dbl>, cate_lbc <dbl>, cate_la <dbl>, cate_lam <dbl>,
## #   cate_lac <dbl>, cate_d <dbl>, cate_dm <dbl>, cate_dc <dbl>,
## #   cate_dl <dbl>, cate_dlm <dbl>, cate_dlc <dbl>, ruvbn_ <dbl>,
## #   ruvbnl_ <dbl>, ruvbnn_ <dbl>, ruvb_ <dbl>
```

2. (4 pts) Convert this data frame so in addition to the six conditions columns, one column contains the mean/variance method combination for each iteration, and another column contains the mse for that iteration's mean/variance method combinations.

```
msemat %>%
  gather(ols_o:ruvb_, key = "mean_var", value = "mse") ->
  msemat
head(msemat)
```

```
## # A tibble: 6 x 8
##   current_seed nullpi Nsamp ncontrols cor          poisthin mean_var   mse
##          <dbl>  <dbl> <dbl>     <dbl> <chr>        <lgl>    <chr>    <dbl>
## 1         2223    0.9    20       100 (0.25, 0.25) TRUE     ols_o    0.197
## 2         2223    0.9     6       100 (0.5)        TRUE     ols_o    0.460
## 3         2223    0.9     6       100 (0.5, 0.5)   TRUE     ols_o    0.664
## 4         2224    0.9     6       100 (0.25, 0.25) TRUE     ols_o    0.433
## 5         2224    0.9    20       100 (0.5, 0.5)   TRUE     ols_o    0.182
## 6         2224    0.9    10       100 (0.5, 0.5)   TRUE     ols_o    0.449
```

3. (4 pts) Create two new columns in the dataframe: one for the mean method used and one for the variance method used. For `ruvbn`, `ruvbnl`, `ruvbnn`, and `ruvb` the variance method should be the empty character `""`.

```
msemat %>%
  separate(col = mean_var, into = c("mean", "var"), sep = "_") ->
  msemat
head(msemat)
```

```
## # A tibble: 6 x 9
##   current_seed nullpi Nsamp ncontrols cor        poisthin mean  var     mse
##          <dbl>  <dbl> <dbl>     <dbl> <chr>      <lgl>    <chr> <chr> <dbl>
## 1         2223    0.9    20       100 (0.25, 0.~ TRUE     ols   o     0.197
## 2         2223    0.9     6       100 (0.5)      TRUE     ols   o     0.460
## 3         2223    0.9     6       100 (0.5, 0.5) TRUE     ols   o     0.664
## 4         2224    0.9     6       100 (0.25, 0.~ TRUE     ols   o     0.433
## 5         2224    0.9    20       100 (0.5, 0.5) TRUE     ols   o     0.182
## 6         2224    0.9    10       100 (0.5, 0.5) TRUE     ols   o     0.449
```

4. (6 pts) Clean up the `cor` variable in the dataframe:

    1. Remove all parentheses from `cor`.
    2. Replace the first number with that number followed by an underscore (you'll probably need back-referencing here).
    3. Remove all commas and spaces.

    For example, `"(0.25, 0.25)"` should turn into `"0.25_0.25"` while `"(0.25)"` should turn into `"0.25_"`.

```
msemat %>%
  mutate(cor = str_replace_all(cor, "[()]", "")) %>%
  mutate(cor = str_replace(cor, "([\\d\\.]+)", "\\1_")) %>%
  mutate(cor = str_replace_all(cor, ",\\s", "")) ->
  msemat
head(msemat)
```

```
## # A tibble: 6 x 9
##   current_seed nullpi Nsamp ncontrols cor       poisthin mean  var     mse
##          <dbl>  <dbl> <dbl>     <dbl> <chr>     <lgl>    <chr> <chr> <dbl>
## 1         2223    0.9    20       100 0.25_0.25 TRUE     ols   o     0.197
## 2         2223    0.9     6       100 0.5_      TRUE     ols   o     0.460
## 3         2223    0.9     6       100 0.5_0.5   TRUE     ols   o     0.664
## 4         2224    0.9     6       100 0.25_0.25 TRUE     ols   o     0.433
## 5         2224    0.9    20       100 0.5_0.5   TRUE     ols   o     0.182
## 6         2224    0.9    10       100 0.5_0.5   TRUE     ols   o     0.449
```

5. (4 pts) Use the results from part 4 to create two new columns in the dataframe: one with the first correlation and one with the second correlation. If `cor` only has one value, then the second correlation should be `NA`. Make sure the new correlation columns are numerics.

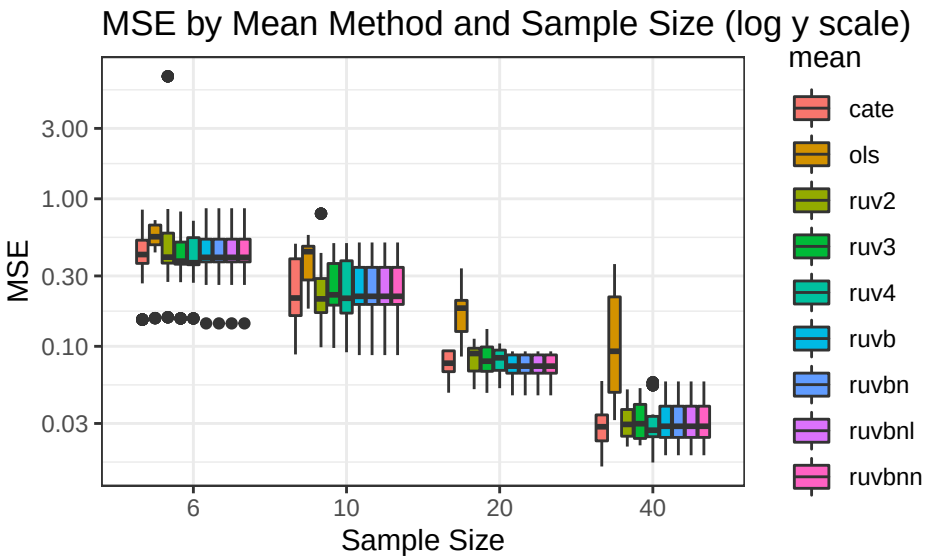
```
msemat %>%
  separate(col = "cor", into = c("cor1", "cor2"), sep = "_") %>%
  mutate(cor1 = parse_number(cor1),
         cor2 = parse_number(cor2)) ->
  msemat
head(msemat)
```

```
## # A tibble: 6 x 10
##   current_seed nullpi Nsamp ncontrols  cor1  cor2 poisthin mean  var
##          <dbl>  <dbl> <dbl>     <dbl> <dbl> <dbl> <lgl>    <chr> <chr>
## 1         2223    0.9    20       100  0.25  0.25 TRUE     ols   o
## 2         2223    0.9     6       100  0.5   NA   TRUE     ols   o
## 3         2223    0.9     6       100  0.5   0.5  TRUE     ols   o
```

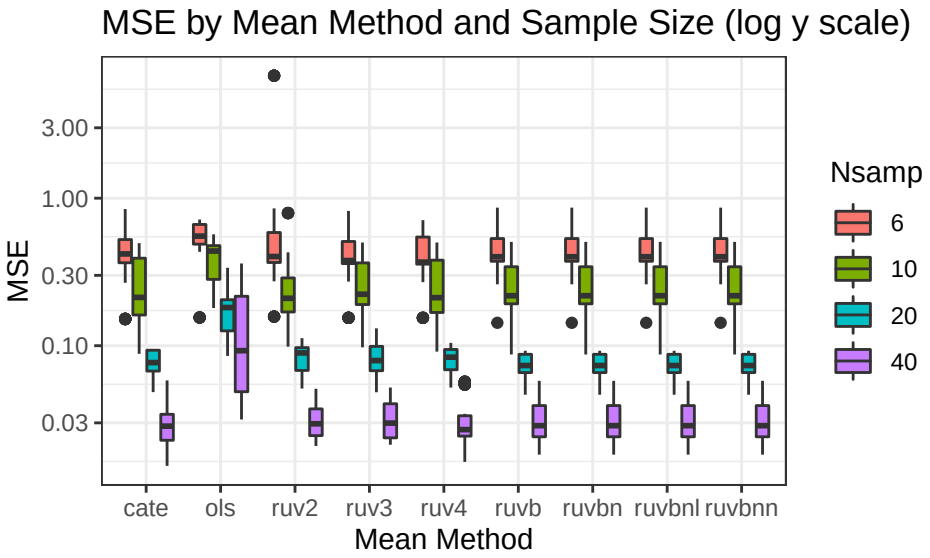
```
## 4         2224    0.9     6       100  0.25  0.25 TRUE     ols   o
## 5         2224    0.9    20       100  0.5   0.5  TRUE     ols   o
## 6         2224    0.9    10       100  0.5   0.5  TRUE     ols   o
## # ... with 1 more variable: mse <dbl>
```

6. (2 pts) Make an appropriate plot to explore the effect of `Nsamp` and the mean method on mean squared error.

```
msemat %>%
  mutate(Nsamp = as_factor(Nsamp)) %>%
  ggplot(aes(y = mse, x = Nsamp, fill = mean)) +
  geom_boxplot() +
  scale_y_log10() +
  xlab("Sample Size") +
  ylab("MSE") +
  ggtitle("MSE by Mean Method and Sample Size (log y scale)") +
  theme_bw()
```




```
  # or
msemat %>%
  mutate(Nsamp = as_factor(Nsamp)) %>%
  ggplot(aes(y = mse, x = mean, fill = Nsamp)) +
  geom_boxplot() +
  scale_y_log10() +
  xlab("Mean Method") +
  ylab("MSE") +
  ggtitle("MSE by Mean Method and Sample Size (log y scale)")+
  theme_bw()
```


MSE by Mean Method and Sample Size (log y scale)
MSE
3.00
1.00
0.30
0.10
0.03
cate
ols
ruv2
ruv3
ruv4
ruvb
ruvbn
ruvbnl
ruvbnn
Mean Method
Nsamp
6
10
20
40